WEEK-3

SESSION-11:

TASK-1:JAVA DATE AND TIME

```
class Result{
 public static String findDay(int month, int day, int year) {
        LocalDate ld=LocalDate.of(year, month, day);
        return ld.getDayOfWeek().toString(); }}
public class Solution {
   public static void main(String[] args) throws IOException {
      BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
      BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));
      String[] firstMultipleInput =
bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");
      int month = Integer.parseInt(firstMultipleInput[0]);
      int day = Integer.parseInt(firstMultipleInput[1]);
      int year = Integer.parseInt(firstMultipleInput[2]);
      String res = Result.findDay(month, day, year);
      bufferedWriter.write(res);
      bufferedWriter.newLine();
      bufferedReader.close();
      bufferedWriter.close(); }}
```

TASK-2:NO.OF DAYS BETWEEN TWO DATES

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
class Solution {
    public int daysBetweenDates(String date1, String date2) {
        DateTimeFormatter fmt=DateTimeFormatter.ofPattern("yyyy-mm-dd");
        LocalDate dates1=LocalDate.parse(date1);
        LocalDate dates2=LocalDate.parse(date2);
        int difference=(int)ChronoUnit.DAYS.between(dates2, dates1);
        return Math.abs((int) difference);
    }
    public static void main(String a[]){
        Solution obj=new Solution();
        System.out.println(obj.daysBetweenDates("2019-06-29","2019-06-30"));
    }
}
```

SESSION-12:

TASK-1:DAY OF THE YEAR

```
import java.time.*;
class Solution {
    public int dayOfYear(String date) {
        LocalDate ld=LocalDate.parse(date);
        return ld.getDayOfYear();
    }
    public static void main(String a[]){
        Solution obj=new Solution();
        int result=obj.dayOfYear("2026-08-04");
        System.out.println(result);
    }
}
```

TASK-2:DAY OF THE WEEK

```
import java.time.*;
import java.time.format.TextStyle;
import java.util.Locale;
class Solution {
    public String dayOfTheWeek(int day, int month, int year) {
        LocalDate ld=LocalDate.of(year,month,day);
        return ld.getDayOfWeek().getDisplayName(TextStyle.FULL,
Locale.ENGLISH);
    }
```

```
    public static void main(String a[]){
        Solution obj=new Solution();
        String result=obj.dayOfTheWeek(31,8,2019);
        System.out.println(result);
    }}
```

SESSION-13:

TASK-1:JAVA PRIORITY QUEUE

```
import java.util.PriorityQueue;
class Student implements Comparable<Student> {
    private int id;
    private String name;
    private double cgpa;

    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    public int getID() {
        return id;
    }

    public String getName() {
```

```
        return name;
    }

    public double getCGPA() {
        return cgpa;
    }

    @Override
    public int compareTo(Student other) {
        if (Double.compare(other.cgpa, this.cgpa) != 0) {
            return Double.compare(other.cgpa, this.cgpa);
        } else if (!this.name.equals(other.name)) {
            return this.name.compareTo(other.name);
        } else {
            return Integer.compare(this.id, other.id);
        }
    }
}

public class Solution {
    private final static Scanner scan = new Scanner(System.in);
    private final static Priorities priorities = new Priorities();

    public static void main(String[] args) {
```

```
        int totalEvents = Integer.parseInt(scan.nextLine());
        List<String> events = new ArrayList<>();

        while (totalEvents-- != 0) {
            String event = scan.nextLine();
            events.add(event);
        }

        List<Student> students = priorities.getStudents(events);

        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student st: students) {
                System.out.println(st.getName());
            }
        }
    }
}
```

TASK-2:JAVA ARRAYLIST

```
public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
```

```
    ArrayList<ArrayList<Integer>> lines = new
ArrayList<ArrayList<Integer>>();
        for (int i = 0; i < n; i++) {
            int d = scan.nextInt();
            ArrayList<Integer> row = new ArrayList<Integer>();
            for (int j = 0; j < d; j++) {
                row.add(scan.nextInt());
            }
            lines.add(row);
        }

        int q = scan.nextInt();
        for (int i = 0; i < q; i++) {
            int x = scan.nextInt() - 1; // 1-based indexing to 0-based
            int y = scan.nextInt() - 1;

            try {
                System.out.println(lines.get(x).get(y));
            } catch (Exception e) {
                System.out.println("ERROR!");
            }
        }

        scan.close();
```

```
    }
}
```

SESSION-14:

TASK-1:LARGEST NUMBER

```
import java.util.Arrays;
import java.util.Comparator;
class Solution {
    public String largestNumber(int[] nums) {
        String[] asStrs = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            asStrs[i] = String.valueOf(nums[i]);
        }
        Arrays.sort(asStrs, new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                String order1 = b + a;
                String order2 = a + b;
                return order1.compareTo(order2);
            }
        });
        if (asStrs[0].equals("0")) {
            return "0";
        }
        StringBuilder largestNumberstr = new StringBuilder();
```

```
        for (String numAsStr : asStrs) {
            largestNumberstr.append(numAsStr);
        }

        return largestNumberstr.toString();
}
}
```

SESSION-15:

TASK-1:SORT CHARCTERS BY FREQUENCY

```
import java.util.*;
class Solution {
    public String frequencySort(String s) {
        Map<Character,Integer> count=new HashMap<>();
        for(char c:s.toCharArray()){
            count.put(c, count.getOrDefault(c, 0) + 1); }
     List<Character> list = new ArrayList<>(count.keySet());
        list.sort((a, b) -> count.get(b) - count.get(a));
        StringBuilder result = new StringBuilder();
        for (char c : list) {
            for (int i = 0; i < count.get(c); i++) {
                result.append(c);}}
        return result.toString();
    }}
```